Handoff 11: Browser behaviour checks (stage 11: ENG-03)

Date: 2026-10-26 (America/Toronto)
Written by: Claude Code (implementer), overnight session
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request, not merged, not deployed. **No database migration.** No unit 2 content; nothing changes for learners yet.

> **How to use this file:** upload it with `build.md`, `docs/plan/browser-checks.md` (the contract), `docs/plan/course-catalog.md` section 7.3 and `docs/plan/lesson-feedback.md`. The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing Python Academy stage 11 (ENG-03): a contract that lets
lessons in unit 2 and later be checked by running the learner's code in
their own browser (Pyodide worker) against test cases from the content
file, while the API compares the reported output and never runs learner
code. I'm attaching build.md, docs/plan/browser-checks.md and this
handoff. Check that the server cannot be made to run code, that a hidden
case's expected output cannot reach the browser, that endless loops and
runaway output are stopped, that learner code has no network, and that
unit 1 is unchanged. Question the decision to send hidden cases' inputs
to the browser. Explain at a first-year computer engineering level.
```

1. Where we are

Stage	What	Status
9	Teaching design and lesson feedback (docs)	Merged (PR #15)
10	ASSESS-07 honest unit 1 checks	PR #16, open. This PR does **not** depend on it, and the two merge cleanly (section 6)
11	ENG-03 browser behaviour checks	**in_review**: this PR

2. Handoff record (build.md template)

Field	Value
Task	ENG-03: content schema for check cases; the worker runs them with a time limit, an output cap and no network; the API returns the existing result states plus the failing case without revealing hidden data; fields for ENG-04; tests; a test-only fixture lesson
Human goal	Unit 2 and later can have trustworthy-for-practice checks of real behaviour (several inputs, hidden cases), without ever running learner code on the server
Relevant build.md sections	11 (runtimes and assessment), 12 (quality gates); `course-catalog.md` section 7.3 (decision 4)
Repository and branch	`torontojohn/python-academy`, branch `eng/browser-checks`
Base commit	`f7bfa28` (`main` after PR #15)
Implementer / reviewer	Claude Code / owner, plus ChatGPT or Claude chat
Deployment status	Not deployed; not merged
New dependencies	**None**
Database migration	**None**

Files changed

File	Change
`backend/app/content.py`	New `CheckCase` and `BrowserCheck` models, the `check` field on challenges, the construct list, and `check_errors()` validation at load time. The existing `VALIDATORS` line is untouched, so it does not collide with PR #16
`backend/app/browser_checks.py`	New. The check plan, the construct finder (parsing only), output comparison, and `judge()`, which turns the browser's reports into a result
`backend/app/main.py`	`GET /api/checks/{lesson_id}`; `/api/check` and `/api/attempts` accept `runs` and use `judge()` for browser-checked lessons. Unit 1 still goes through `assess()` unchanged
`frontend/public/python-worker.js`	One line routes `mode: 'cases'` to the new `runCases`; the unit 1 path below it is byte-for-byte unchanged

File	Change
`frontend/public/check-runner.js`	New. Starts the worker, enforces the time limit by terminating it, returns one report per case; `forSubmission()` drops Python's message before sending
`backend/tests/fixtures/browser_checks/`	New. The **TEST FIXTURE** lesson `fixture.sum-two`, outside `backend/content/`, so it is never served to learners. It has a README saying so
`backend/tests/test_browser_checks.py`	New: 42 tests (section 5)
`frontend/tests/browser-checks.spec.ts`	New: 9 Playwright tests running the real worker (section 5)
`backend/tests/test_sessions.py`	The route list includes the new public `GET /api/checks/{lesson_id}`
`docs/plan/browser-checks.md`	New. The contract: schema, flow, result fields, what "hidden" means, limits
`backend/content/README.md`, `build.md`	Short pointers to the new contract

Not changed, as required: the worker's enforced CSP and the page CSP (`frontend/next.config.mjs`, `frontend/proxy.ts`, `frontend/csp.mjs`), rate limits (`backend/app/limits.py`), the unit 1 checker (`backend/app/lesson_checks.py`), migrations, and `docs/plan/backlog.md`. The diff confirms all of these files are untouched.

3. How it works (short version; full contract in `docs/plan/browser-checks.md`)

1. **Content** declares cases: inputs for `input()`, expected output, visible or hidden, and a description shown on failure. The challenge can also list required and forbidden constructs and a time limit per case. The API validates all of this at start-up. A hidden case's description must not contain its inputs or expected output.
2. **The browser** fetches `GET /api/checks/<lesson>`, which includes inputs for every case but expected output only for visible cases. It then runs the code in a fresh worker: - once per case, in a fresh namespace - output capped at 10,000 characters - network APIs removed after Pyodide loads
3. **The runner** terminates the worker if a case runs past its time limit. That case becomes `timeout` and the remaining cases `not_run`.
4. **The API** compares each reported output with the expected output. It parses the code with `ast.parse` for constructs and syntax errors; parsing does not run anything. It returns `passed`, `needs_work`, `execution_error`, `unsupported` or `service_error`, plus per-case statuses and a `failed` object.
5. **For ENG-04**, `failed` holds the case's description, `reason`, `error_type` and `line`. For a visible case it also holds the first different line, with expected and actual text. Python's full message stays in the browser report, for display only.

Example messages, from the real end-to-end run in section 5: - "For the numbers 2 and 3, line 1 should be "5", but your program printed "-1"." - "Your program printed something different for two larger numbers (a

hidden test case). Check that it works from the replies it is given, not from fixed values." - "Your program was still running after 3 seconds on the numbers 2 and 3, so it was stopped. Check for a loop that never ends."

4. Decisions and assumptions (please review)

1. **Hidden cases' inputs go to the browser; their expected output never does.** This is the one place the task could not be met literally. - The code runs in the learner's browser, and a case cannot run without its inputs. So "hidden tests are not leaked in any API response" is implemented as follows. - The hidden **expected output** appears in **no** API response. A test checks every GET route, signed out and as a student, and passing and failing posts to both check endpoints. - Hidden **inputs** appear **only** in `GET /api/checks/<lesson>`, also tested. - Knowing an input doesn't give away the answer: you still have to compute the right output, which is the task. - The alternative that hides inputs too is a hosted runner on a server, which is deferred for cost and isolation reasons. - **Question 1** asks whether you accept this.
2. **The API decides the result, not the worker.** The task said the worker returns the result states. The worker cannot do that for hidden cases without their expected output, which would break point 1. So the worker and runner report what happened in each case, and the API returns the result states and the failing case. A single judge also keeps preview checks and saved attempts identical.
3. **Required and forbidden constructs are per challenge, not per case**, and are checked by parsing on the server. They describe the program, not one run.
4. **Practice-level evidence, not tamper-proof.** A learner who edits what their browser sends can claim any output, as with unit 1 today (`course-catalog.md` section 7.3).
5. **Limits:** - time 3 seconds per case by default (content may set 0.5 to 10) - Python load 30 seconds - output 10,000 characters per case - at most 10 cases - comparison ignores trailing spaces, trailing blank lines and Windows line endings
6. **The new endpoint is public**, like the lessons and preview checks. It is a GET, so it has no rate limit, consistent with the other GETs.
7. **What a saved attempt stores:** for browser-checked lessons, the `output` the page sends (proposed: the first case's output). Per-case reports are not stored. No new column, so no migration.

5. Tests run (exact commands and results)

From the repository root, on macOS (Python 3.12.14 in `backend/.venv`, Node 24):

Command	Result
`cd backend && .venv/bin/python -m pytest -q tests/test_browser_checks.py`	`42 passed`
`cd backend && .venv/bin/python -m pytest -q`	`219 passed, 9 skipped` (the 9 need PostgreSQL)

Command	Result
Same with `TEST_POSTGRES_URL` set to a local throwaway PostgreSQL 16	`228 passed`
`cd frontend && npx tsc --noEmit`	No errors
`cd frontend && npm run build`	Compiled successfully
`cd frontend && CI=1 npx playwright test tests/browser-checks.spec.ts`	`9 passed`
`cd frontend && CI=1 npm run test:e2e` (everything)	`39 passed`
Backend tests on a temporary merge of this branch with PR #16	`255 passed, 9 skipped`

What the tests cover

Requirement	Tests
Schema validation	The fixture loads. 14 kinds of invalid check stop content loading: a validator without a check or the reverse, an unknown version, duplicate case IDs, no visible case, an unknown construct, require and forbid overlapping, a hidden description showing an input, a hidden description showing the expected output, time limit, input length, too many cases, an unknown field, and expected output length. Real content still loads, and unit 1 has no browser check
A correct solution passes	Backend: correct reports pass, with per-case statuses. Playwright: real Pyodide prints the expected output for all three cases, hidden included
A wrong one fails with a useful message	Backend: the first different line on a visible case, missing and extra lines, and a generic message for a hidden case. Playwright: real wrong output
An infinite loop is stopped	Playwright: `while True: pass` gives `timeout` and two `not_run`, and a fresh worker works afterwards. Backend: `execution_error` with the right message
Runaway output is capped	Playwright: the output is cut at 10,000 characters and reported as `output_limit` for each case. Backend: `execution_error`; reports over 10,000 characters are rejected with 422
Hidden tests not leaked	Backend: the plan has no hidden expected output; across 30+ responses (every GET route signed out and signed in, and both check endpoints with passing and failing reports) the hidden expected output never appears, and hidden inputs appear only in the plan
No network	Playwright: `js.fetch`, `js.XMLHttpRequest`, `js.importScripts` and `pyodide.http.open_url` all fail inside learner code
Fields for ENG-04	Backend: `failed` carries description, reason, error type and line. Playwright: real `NameError` on line 2, `SyntaxError` on line 1, `EOFError` on line 3, and Python's message kept out of `forSubmission()`
Server never runs code	Backend: code that would write a file is judged, and no file appears

Requirement	Tests
Unit 1 unchanged	Backend: a unit 1 preview check returns the original four fields. All 30 existing Playwright tests, including real Python for every unit 1 lesson, still pass
Other	Each case starts with fresh variables; output with no final newline is reported; constructs required or forbidden; reports that don't match the plan, or Python unavailable, give `service_error`; saved attempts use the same judgement and award XP once

Controls (the tests catch mistakes)

Deliberate break	Result
The plan includes hidden expected output	2 tests **failed** (the plan test and the all-responses leak test)
The hidden failure message includes the expected output	2 tests **failed**
Content validation stops checking hidden descriptions	2 tests **failed**
The judge treats a reported timeout as fine	1 test **failed**

Each break was undone afterwards.

The whole chain, with no mocks

I ran the real API locally with the fixture lesson added in memory (a scratch launcher, not committed), the production frontend build, and Chromium. The page fetched the plan, ran the runner and real Pyodide, and posted the reports to `/api/check`:

Program	Case statuses	Result
Correct	ok, ok, ok	`passed`
Subtracts instead of adding	ok, ok, ok	`needs_work`: line 1 should be "5", but your program printed "-1"
Hard-codes the two visible cases	ok, ok, ok	`needs_work`, caught **only by the hidden case**
Endless loop	timeout, not_run, not_run	`execution_error` after 3 seconds
Prints forever	output_limit ×3	`execution_error`
Uses an undefined name	error ×3	`execution_error`: NameError on line 2

There were no CSP reports during the run.

6. Open PRs and conflicts

- **PR #16 (ASSESS-07)** edits `content.py`'s `VALIDATORS` line, unit 1 content and `lesson_checks.py`. This PR leaves all of those lines alone. `git merge-tree` shows no conflict, and the backend suite passes on the merged result (255 passed). Either can merge first.
- **The backlog was not edited**, as instructed. Changes to make when convenient:
- **ENG-03** → `in_review (stage 11, handoff 11)`, then `verified` after the live check in section 8
- Stage table: add "11. ENG-03 | Browser behaviour checks | in_review (handoff 11)"
- **ENG-04** can note that it uses `failed` from `docs/plan/browser-checks.md` section 4
- **UI-05** (lesson page reads lessons per unit) is now the step that connects the lesson page to this contract, together with ENG-04
- `course-catalog.md` section 7.3: "Add behaviour checks run in the learner's browser" is built (ENG-03)

7. Mocks and unverified behaviour

- **Backend judge tests** post hand-written case reports. They have the same shape the runner produces, which the Playwright tests and the end-to-end run confirm, but they are not produced by Pyodide.
- **Playwright tests** run the real worker and real Pyodide from the CDN under the real CSP, on a real app page (the 404 page, which makes no API calls). They do not call the API. The plan is built in the test from the fixture, with expected outputs removed, exactly as the endpoint serves it.
- **The fixture lesson** reaches the API only in tests (`monkeypatch` into `CONTENT`) and in the local scratch run. It is not in `backend/content/`, so production never loads it.
- **Not verified:** Safari and Firefox (Chromium only); timing on slow devices and connections (the 30-second load limit and the 3-second per-case default are guesses to revisit with unit 2); and the live site, since no learner-visible page uses this yet.

8. Live checks for you after merging

Nothing changes for learners: no lesson uses browser checks yet. These checks confirm that the deploy is healthy and that the new endpoint exposes nothing.

- [ ] **Deploy finished.** Both services show the new deploy as live. The API log still says `Database schema is at migration 0006`, or the latest number if PR #16 merged first; there is no new migration in this PR.
- [ ] **The new endpoint exists and serves nothing yet:**

```sh
curl -s -o /dev/null -w "%{http_code}\n" https://python-academy-dji8.onrender.com/api/checks/hello
```

prints `404`. Unit 1 lessons have no browser check, and the test fixture is not deployed. - [ ] **Unit 1 still works exactly as before.** Open `/preview?lesson=hello&card=5`, run and check a correct answer, and it passes as usual. Do one signed-in challenge as a student: XP and badge as before. - [ ] **The runner file is served:**

```
sh curl -sI https://python-academy-dji8.onrender.com/check-runner.js | head -1
```

shows `HTTP/2 200`. - [ ] **Worker policy unchanged:**

```
sh curl -sI https://python-academy-dji8.onrender.com/python-worker.js | grep -i "^content-security-policy:"
```

shows the same `default-src 'none'; script-src 'unsafe-eval' https://cdn.jsdelivr.net; connect-src https://cdn.jsdelivr.net` as before. - [ ] **Optional, to see it work in your browser.** Open `https://python-academy-dji8.onrender.com/no-such-page`, then DevTools → Console, and paste:

```
js const r = await import('/check-runner.js'); await r.runCases('while True:\n pass', {time_limit_ms: 2000, output_limit: 10000, cases: [{id: 'a', inputs: []}]});
```

After Python loads and about 2 more seconds, it returns `[{id: 'a', status: 'timeout', …}]`, and the page stays responsive.

9. Questions for the owner

1. **Hidden inputs:** accept that a hidden case's inputs reach the browser, while its expected output never does (section 4, point 1)? The alternative is a hosted runner, which costs money and needs its own isolation.
2. **Limits:** are 3 seconds per case and 10 cases per check right for unit 2? Content can lower or raise the time to between 0.5 and 10 seconds.
3. **Saved attempts:** is storing only the code and the first case's output enough? Per-case reports would need a migration.

10. Next concrete action

The owner reviews this PR and PR #16 and answers section 9. Then ENG-04 (the feedback engine) and UI-05 connect the lesson page to this contract, and unit 2 (EDU-04) can be written with browser checks from the start.